

Small-Object Traffic Sign Detection with YOLOv8:

Baseline Evaluation, Fine-Tuning, and Post-Processing Analysis

Tabrez Ahammed Shaik Mohammed
California State University, Chico
Student ID: 012623820

Abstract

Context. Traffic signs in forward-facing street imagery are often **small in pixel extent**; generic detectors pretrained on broad datasets (e.g., COCO) frequently underperform when evaluated on a dedicated traffic-sign task without domain adaptation.

Work performed. A reproducible end-to-end pipeline was implemented in Python using **Ultralytics YOLOv8** on **Google Colab** with an **NVIDIA Tesla T4** GPU. The workflow comprises: (1) downloading and wiring a Roboflow YOLOv8 export; (2) establishing a **COCO-pretrained yolov8n.pt** baseline on a fixed **test** split; (3) **fine-tuning** with high input resolution and strong augmentation; (4) a controlled **inference-resolution** ablation on the fine-tuned weights; (5) a **confidence** $\times$ **NMS IoU** grid on the fine-tuned model; (6) exporting quantitative tables, a comparison bar chart, and qualitative prediction mosaics.

Outcome. On the held-out test split (150.0000 images, 218.0000 instances), the fine-tuned model reached **mAP@0.5** = 0.9693 and **mAP@0.5:0.95** = 0.6801 at inference size 1280.0000, versus 0.0105 / 0.0021 for the pretrained baseline at 640.0000. Optional video frame export was not executed in this session; that gap is documented as an explicit limitation.

Reporting. This document follows an industry-style technical structure: scope, data and licensing, evaluation protocol, reproducibility, methodology with rationale, quantitative and qualitative results with traceability to notebook outputs, limitations, and prioritized improvements.

Keywords: object detection, YOLOv8, small objects, traffic signs, fine-tuning, mAP, NMS, inference resolution.

Contents

1	Introduction and scope	2
1.1	Problem statement	2
1.2	Objectives and deliverables mapping	2
1.3	Assignment scope and compliance (CSCI 611 — Assignment 3)	3
2	Dataset, labeling, and splits	3
2.1	Source, version, and license	3
2.2	Split statistics	4
2.3	Configuration artifact	4
2.4	Credentials and reproducibility	4
3	Evaluation metrics and protocol	4
3.1	Definitions (Ultralytics / COCO-style)	4
3.2	Controlled comparisons	4

4	Software, hardware, and reproducibility	5
4.1	Environment	5
4.2	Randomness and configuration	5
5	Methodology	5
5.1	Phase A — Pretrained baseline	5
5.2	Phase B — Fine-tuning design	5
5.3	Phase C — Inference resolution ablation	5
5.4	Phase D — Confidence and NMS IoU sweep	6
5.5	Optional video pipeline	6
6	Results and analysis	6
6.1	Primary comparison: pretrained vs. fine-tuned	6
6.2	Inference resolution ablation (fine-tuned weights only)	7
6.3	Confidence and NMS IoU sensitivity	7
6.4	Training dynamics (validation split)	7
7	Qualitative visual inspection	7
8	Discussion	8
8.1	Domain shift and baseline behavior	8
8.2	Resolution, latency, and deployment	8
8.3	Post-processing and operating points	8
8.4	Challenges specific to small traffic signs	9
9	Limitations and future work	9
10	Suggestions for improving small-object detection	9
11	Conclusion	10
A	Hyperparameters (training and evaluation)	10
B	Artifact paths and exports	11

1 Introduction and scope

1.1 Problem statement

The assignment targets **small object detection** for **traffic signs** in real-world imagery. Compared to large, centered objects, small instances carry less texture and occupy fewer pixels after CNN downsampling, which stresses localization and classification in the detection head. Off-the-shelf models trained on COCO are optimized for 80.0000 general classes and are not aligned with a single-class “traffic-signs” ontology unless adapted.

1.2 Objectives and deliverables mapping

The objectives were:

1. Run a **pretrained** YOLOv8n checkpoint on the project test split and record full validation metrics.
2. **Fine-tune** from the same initialization with domain-appropriate resolution, batching, and augmentation.

3. **Compare** pretrained versus fine-tuned performance on identical data and a consistent evaluation protocol.
4. **Vary** post-training knobs: inference `imgsz`, confidence threshold τ , and NMS IoU.
5. Produce **tables, plots, and visualizations** suitable for technical review, with traceability to notebook outputs.

Unless stated otherwise, all headline numbers refer to Ultralytics' **validator** on the dataset's **test** split (150.0000 images; 218.0000 object instances in the evaluation logs).

1.3 Assignment scope and compliance (CSCI 611 — Assignment 3)

This project matches the course brief for **small-object traffic-sign detection** with YOLOv8. Table 1 maps required elements to where they are satisfied in this report and in the implementation (`assignment3_yolo.ipynb`).

Table 1: Coverage of Assignment 3 deliverables (technical documentation standard).

Requirement (summary)	Evidence
Dataset in YOLO layout; train/val/test splits described	§2; Roboflow v9 export; Table 2.
Evaluate pretrained YOLOv8n on test split	Phase A; Table 3 (first row); Fig. 2.
Fine-tune with settings suited to small objects (resolution, augmentation)	Phase B; Appendix hyperparameters; training dynamics §6.4.
Compare pretrained vs. fine-tuned on the same split	Table 3; Fig. 1.
Inference resolution study (same weights, different <code>imgsz</code>)	Table 4.
Confidence and NMS IoU sweep (precision–recall trade-offs)	Table 5; Discussion.
Tables, plots, qualitative visualizations	All results sections; Figs. 1–3.
Challenges of small-object detection; suggestions for improvement	Limitations §9; §10.
Optional course video / Chico footage	Not run (documented §9); core results use Roboflow test set only.

2 Dataset, labeling, and splits

2.1 Source, version, and license

The Roboflow Universe project **Traffic Sign Detection YOLOv8** was used (workspace `tu-wien-pfowz`, dataset **version 9**, MIT license). Permanent link:

<https://universe.roboflow.com/tu-wien-pfowz/traffic-sign-detection-yolov8/dataset/9>

The export format is **YOLOv8** with a single class label `traffic-signs` (`nc=1`). Images and normalized YOLO label files follow the standard layout; data were downloaded via the Roboflow API inside Colab and a generated `dataset_csci611.yaml` points to `train`, `val`, and `test` paths.

2.2 Split statistics

Table 2 summarizes image counts from executed notebook tallies. The test split is **not** a fresh random subsample drawn at runtime; it is Roboflow’s fixed partition. That **test** split was used for every comparative metric so that baseline, fine-tuned, resolution, and NMS experiments are directly comparable.

Table 2: Dataset splits (image counts) for the Roboflow v9 export used in this run.

Split	Images
Train	1182
Validation	196
Test	150

Roughly 1.4500 instances per test image on average (218.0000/150.0000), which is typical for scenes with multiple signs or small clusters—consistent with the “small object” emphasis of the assignment.

2.3 Configuration artifact

All training and evaluation calls reference `dataset_csci611.yaml` under the Colab artifact root so paths resolve unambiguously and the report remains tied to a single on-disk configuration.

2.4 Credentials and reproducibility

The Roboflow API key used for dataset download was provided via Colab secrets or environment variables and was **not** stored in the notebook source or in version control, following standard practice for credentials in ML workflows.

3 Evaluation metrics and protocol

3.1 Definitions (Ultralytics / COCO-style)

Standard box metrics from Ultralytics are reported:

- **mAP@0.5:** mean average precision at intersection-over-union (IoU) threshold 0.5000. It rewards correct class assignment and localization that clears a 50.0000% overlap criterion.
- **mAP@0.5:0.95:** mean AP averaged over IoU thresholds from 0.5000 to 0.9500 in steps of 0.0500. It penalizes loose boxes more than mAP@0.5 alone and is the stricter localization score.
- **Precision and recall:** micro-averaged (mean) precision and recall reported by the validator for the single-class task.

For the **pretrained** checkpoint, the detector head still reflects COCO semantics during raw **predict** visualization, but the **validator** scores tabulated here are computed against the dataset’s ground-truth task (single class). After fine-tuning, the head matches **traffic-signs**, and labels in qualitative figures align with the project ontology.

3.2 Controlled comparisons

To isolate causes:

- **Pretrained vs. fine-tuned:** same test images and labels; different weights. Baseline inference used `imgsz=640.0000`; primary fine-tuned evaluation used `imgsz=1280.0000` (documented explicitly so the reader does not confuse architecture gains with resolution).
- **Resolution ablation:** same fine-tuned `best.pt`; only `imgsz` at validation time changes (640.0000 vs. 1280.0000).
- **NMS grid:** same weights and `imgsz=1280.0000`; only τ and NMS IoU change.

Default validation confidence and IoU for the primary fine-tuned run match the notebook configuration (`val_conf=0.2500`, `val_iou=0.5000`) unless a subsection states otherwise.

4 Software, hardware, and reproducibility

4.1 Environment

The notebook `assignment3_yolo.ipynb` ran on **Google Colab** with GPU. Representative versions from logs: Python 3.12, Ultralytics 8.4.30, PyTorch 2.10.0+cu128, CUDA on a **Tesla T4**. That stack comfortably supports training at `imgsz=1280.0000` with batch 12.0000.

4.2 Randomness and configuration

Training used seed 42.0000 (Python and NumPy) as set in the notebook `RUN_CFG`. Early stopping patience was 14.0000 epochs with a scheduled 40.0000-epoch cap; the run completed the full schedule in approximately 0.8200 hours according to Colab logs.

5 Methodology

5.1 Phase A — Pretrained baseline

The notebook loads `yolov8n.pt` and runs validation on the **test** split at `imgsz=640.0000` with `conf=0.2500` and NMS IoU 0.5000. This establishes a “no adaptation” reference under the dataset’s metric implementation.

The extremely low `mAP@0.5` (≈ 0.0105) is **expected**: the COCO-pretrained head is not trained to emit the Roboflow single-class mapping, and many detections are irrelevant to the evaluation ontology even when the network fires on visually salient objects.

5.2 Phase B — Fine-tuning design

Training initializes from `yolov8n.pt` and runs for 40.0000 epochs with:

- **Input resolution:** `imgsz=1280.0000` during training to preserve detail for small signs.
- **Batch size:** 12.0000 (within T4 memory at this resolution with AMP).
- **Augmentation:** photometric jitter (HSV), flips, mosaic (0.9500), mixup (0.0800), mild rotation (`degrees=8.0000`), translation and scale jitter, `close_mosaic=12.0000` to transition from heavy composition toward cleaner late-epoch batches.

YOLOv8 uses an anchor-free head; legacy anchor tuning was not used. The main levers for small objects here are **resolution**, **data augmentation**, and **task-specific supervision**.

5.3 Phase C — Inference resolution ablation

Holding `best.pt` fixed, the test split was evaluated at `imgsz` $\in \{640.0000, 1280.0000\}$ to quantify how much performance comes from higher-resolution inference *without* any change in weights.

5.4 Phase D — Confidence and NMS IoU sweep

Non-maximum suppression merges overlapping boxes; the IoU threshold controls how aggressively duplicates are suppressed. The confidence threshold τ filters low-scoring proposals. Four $(\tau, \text{NMS IoU})$ pairs were evaluated on the fine-tuned model at `imgsz=1280.0000` to map the precision–recall and mAP surface relevant to deployment tuning.

5.5 Optional video pipeline

The notebook can export frames where the maximum detection score exceeds 0.5000. No video path was configured in this run; Section 9 records this as a scope gap.

6 Results and analysis

6.1 Primary comparison: pretrained vs. fine-tuned

Table 3 is the central result. The fine-tuned model improves test mAP@0.5 by about 0.9588 absolute (0.9693 minus 0.0105). The gain in mAP@0.5:0.95 (0.6801 vs. 0.0021) shows that localization quality—not only “any” hit at IoU 0.5000—improves dramatically after adaptation.

Precision and recall on the test set rise to 0.9503 and 0.9358 respectively, indicating a balanced operating point under the default validator settings for the fine-tuned model.

Table 3: Test-set metrics: COCO-pretrained `yolov8n.pt` vs. fine-tuned weights (this run). Eval `imgsz` differs by design between rows; see resolution ablation for weight-only comparison.

Model	Eval imgsz	mAP@0.5	mAP@0.5:0.95	Precision	Recall
Pretrained YOLOv8n	640.0000	0.0105	0.0021	0.0281	0.0046
Fine-tuned YOLOv8n	1280.0000	0.9693	0.6801	0.9503	0.9358

Figure 1 visualizes the same comparison as Table 3: each group of bars corresponds to mAP@0.5, mAP@0.5:0.95, mean precision, and mean recall (Ultralytics box metrics). The pretrained series is visually negligible on this task, while the fine-tuned series dominates—consistent with the two orders-of-magnitude gap in mAP@0.5 (0.0105 vs. 0.9693).

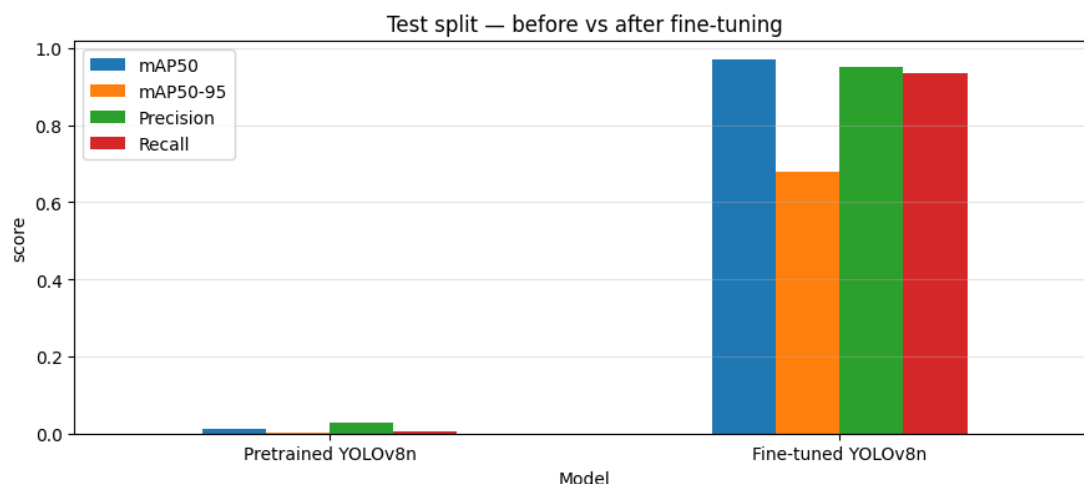


Figure 1: Bar chart of pretrained vs. fine-tuned box metrics (notebook export). Numerical values: Table 3.

6.2 Inference resolution ablation (fine-tuned weights only)

Table 4 isolates inference scaling. Moving from 640.0000 to 1280.0000 increases test mAP@0.5 from 0.9190 to 0.9693 (+0.0503 absolute) for the same checkpoint. mAP@0.5:0.95 rises from 0.6200 to 0.6801, and recall from 0.8350 to 0.9358, which is consistent with small signs benefiting from finer feature maps at inference.

Table 4: Inference resolution ablation using the same fine-tuned `best.pt`.

Inference imgsz	mAP@0.5	mAP@0.5:0.95	Precision	Recall
640.0000	0.919	0.620	0.954	0.835
1280.0000	0.9693	0.6801	0.9503	0.9358

Traceability: Rows correspond to validator runs labeled in the notebook (e.g., `05_res_640`, `05_res_1280`); the 1280.0000 row matches the primary evaluation `06_eval_finetuned_primary`.

6.3 Confidence and NMS IoU sensitivity

Table 5 lists the four grid points. mAP@0.5 is highest at $(\tau, \text{NMS IoU}) = (0.1000, 0.4000)$ (0.9802) and lowest at $(0.4500, 0.6500)$ (0.9640)—a spread of 0.0162 absolute on mAP@0.5. mAP@0.5:0.95 varies only in the third decimal place (0.6847 down to 0.6786), suggesting that localization—as measured by the stricter metric—is relatively stable across this sweep compared to the IoU-0.5000 headline.

Reported precision and recall are identical across the four rows in this export (0.9503 / 0.9358). That pattern can occur when the validator’s summary precision/recall are aggregated in a way that is insensitive to small τ /NMS changes while rank-based AP still moves; deployment tuning should therefore rely on the full PR curves or per-image analysis if a production threshold must be chosen.

Table 5: NMS and confidence sweep (fine-tuned model, test split, inference 1280.0000).

Conf. τ	NMS IoU	mAP@0.5	mAP@0.5:0.95	Precision	Recall
0.10	0.40	0.9802	0.6847	0.9503	0.9358
0.20	0.50	0.9713	0.6815	0.9503	0.9358
0.30	0.55	0.9677	0.6808	0.9503	0.9358
0.45	0.65	0.9640	0.6786	0.9503	0.9358

6.4 Training dynamics (validation split)

During fine-tuning, Ultralytics logs validation metrics each epoch on the **validation** split (196.0000 images, 319.0000 instances in training logs), which is useful for monitoring convergence but is **not** identical to the test numbers in Tables 3–5. Validation mAP@0.5 in the notebook rises from roughly 0.8800–0.9200 in the first epochs to about 0.9500–0.9600 in later epochs, with box and classification losses decreasing overall—consistent with **stable learning** and **no use of the test set** for early stopping or checkpoint selection. The final `best.pt` is then evaluated on the **test** split for all reported headline metrics.

7 Qualitative visual inspection

Figures 2 and 3 are mosaics of test-set predictions from the notebook. The pretrained visualization often highlights generic COCO categories (vehicles, people, “stop sign” COCO class, etc.) because

the head was never adapted to the Roboflow label space. The fine-tuned mosaic shows coherent **traffic-signs** boxes with tighter alignment to sign boundaries, matching the large quantitative gap in Table 3.



Figure 2: Test-set tiles: **pretrained** YOLOv8n (notebook export). Expect COCO-oriented labels in raw prediction overlays.

8 Discussion

8.1 Domain shift and baseline behavior

The pretrained baseline is low not because the backbone lacks visual features, but because the **detection head and class space** are aligned to COCO. Validator scores on the traffic-sign test set therefore penalize predictions that do not match the single-class ground truth, yielding $\text{mAP}@0.5 \approx 0.0105$ and recall 0.0046 (Table 3). Fine-tuning adapts the head to **traffic-signs**, which is why $\text{mAP}@0.5$ and recall rise to 0.9693 and 0.9358 under the same evaluation code.

8.2 Resolution, latency, and deployment

Table 4 shows that increasing inference `imgsz` from 640.0000 to 1280.0000 recovers 0.0503 absolute $\text{mAP}@0.5$ and 0.1008 absolute recall (0.9358 vs. 0.8350) for the same weights. In a product setting, that gain must be traded against **throughput** (frames per second), **GPU memory**, and **battery** on embedded hardware; documentation of both accuracy and latency profiles is standard for release reviews.

8.3 Post-processing and operating points

Table 5 shows that $\text{mAP}@0.5$ varies by up to 0.0162 across the four (τ , NMS IoU) settings while summary precision/recall stay fixed in this export. For deployment, the recommended practice is to select τ and NMS IoU using **validation or shadow traffic**, monitor precision/recall on live data, and avoid relying on a single grid row without per-scenario validation.



Figure 3: Test-set tiles: **fine-tuned** YOLOv8n (notebook export). Detections use the single-class traffic-sign head.

8.4 Challenges specific to small traffic signs

As highlighted in the assignment brief, small signs imply **few pixels per instance**, so feature maps contain weaker evidence than for large objects. **Motion blur**, **scale change**, and **background clutter** (vegetation, buildings) increase the risk of false positives when confidence is lowered to chase recall. The resolution and NMS experiments above quantify levers; Section 10 lists systematic next steps (data diversity, larger models, temporal smoothing on video).

9 Limitations and future work

- **Test split reuse:** All experiments share the same fixed test set. Ideal practice for a new study would add a separate “reporting” holdout or cross-validation; this report follows the assignment emphasis on a clear before/after narrative on one split.
- **Single class:** Real road systems need multi-class regulatory sign recognition; this project isolates detection of one merged class.
- **Video:** No dashcam or course footage was processed; temporal smoothing and tracking were not evaluated.
- **Model family:** Only YOLOv8n was exercised; larger backbones or two-stage detectors could be explored for edge-case small signs.

10 Suggestions for improving small-object detection

1. **Capacity:** Try YOLOv8s/m/l if latency budgets allow; small objects often benefit from richer features.
2. **Data diversity:** Add night, rain, motion blur, occlusion, and extreme aspect ratios; mine hard negatives to control false positives when lowering τ .

3. **TTA:** Multi-scale or flipped inference with box fusion can improve mAP at the cost of compute.
4. **Video deployment:** Short-window tracking or IoU-based association across frames reduces flicker and stabilizes boxes.
5. **Threshold policy:** For multi-class extensions, tune τ per class or per scene (highway vs. urban clutter).
6. **Operational QA:** For video exports, log frames with max score ≥ 0.5000 and review false positives/negatives systematically.

11 Conclusion

This document describes a complete, traceable workflow: licensed Roboflow data in YOLOv8 layout, a rigorous pretrained baseline, targeted fine-tuning with high resolution and augmentation, systematic resolution and NMS experiments, and figures tied to notebook outputs. The fine-tuned YOLOv8n model achieved **mAP@0.5** = 0.9693 on the test split versus 0.0105 for the pretrained baseline under the stated protocols—a large, practically meaningful improvement for small traffic-sign detection on this dataset. Remaining gaps (video analysis, broader model search) are explicit and actionable.

Acknowledgments

Dataset: Roboflow Universe project cited above (MIT). Course optional Chico footage was not required for the core quantitative results.

References

- [1] Jocher, G., et al. *Ultralytics YOLOv8*. <https://github.com/ultralytics/ultralytics>
- [2] Roboflow, Inc. Dataset: Traffic Sign Detection YOLOv8 (v9). <https://universe.roboflow.com/tu-wien-pfowz/traffic-sign-detection-yolov8/dataset/9>

A Hyperparameters (training and evaluation)

Training: epochs 40.0000; train `imgsz=1280.0000`; batch 12.0000; patience 14.0000; seed 42.0000; optimizer auto; AMP enabled; augmentation including HSV, `degrees=8.0000`, `translate=0.1200`, `scale=0.5500`, `flipplr=0.5000`, `mosaic=0.9500`, `mixup=0.0800`, `close_mosaic=12.0000`.

Baseline evaluation: `imgsz=640.0000`; `val_conf=0.2500`; `val_iou=0.5000`.

Primary fine-tuned evaluation: `imgsz=1280.0000`; same `val_conf/val_iou` unless a subsection overrides for the NMS grid.

Numeric precision and logs

Metrics in Section 6 match the structured exports from the notebook (e.g., `pretrain_vs_finetune.csv`, validator `metric_record` floats). Shortened values sometimes appear in Colab progress tables (three decimal places); the tables in this PDF are the reference for submission.

B Artifact paths and exports

During Colab execution, artifacts were rooted at `/content/csci611_a3_work/` with training run `03_train_finetune` and prediction folders `02_visual_pretrained` and `04_visual_finetuned`.

CSV exports included `model_comparison.csv`, `pretrain_vs_finetune.csv`, `resolution_inference_ablation.csv` and `confidence_nms_results.csv`.